

Project Executable Files

Date	2 july 2026
Team ID	xxxxxxx
Project Name	xxxxxxx
Maximum Marks	3 Marks

Project Executable Files

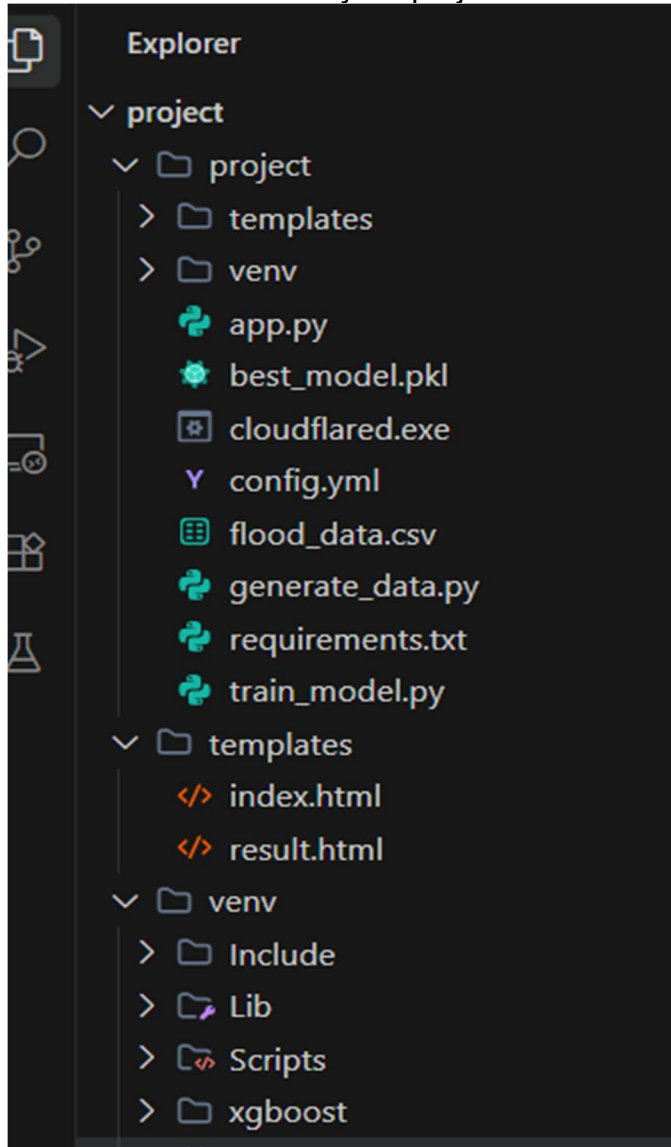
This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	yes
2	README / Setup Guide (instructions to run the project)	yes
3	requirements.txt / package.json / dependency file	yes
4	Database schema / seed files (if applicable)	NA (if you used only a CSV dataset)
5	Environment configuration file (.env.example or similar)	No (or Yes, if used)
6	Deployed application URL (if hosted)	No (or Yes, if deployed on IBM Cloud)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (Web application)
8	Dockerfile / Containerization config (if applicable)	No (unless you created one)
9	Test files and test results	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:



The screenshot shows the VS Code Explorer sidebar with the following structure:

- project
 - project
 - templates
 - venv
 - Include
 - Lib
 - Scripts
 - xgboost
 - templates
 - index.html
 - result.html
 - venv
 - Include
 - Lib
 - Scripts
 - xgboost

Files listed in the project folder:

- app.py
- best_model.pkl
- cloudflared.exe
- config.yml
- flood_data.csv
- generate_data.py
- requirements.txt
- train_model.py

Example text in the editor: `example:]`

Example text in the editor: `package.json for Node)`

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	<i>IBM Cloud URL (if deployed) or Not Deployed</i>
Login Credentials (Demo)	Not Required <i>(if your application does not have a login page)</i>
Platform / Hosting Provider	IBM Cloud <i>(or Local Flask Server if not deployed)</i>
Repository Link	https://github.com/rajkumar0611
Demo Video Link	

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Step 4: Run Instructions

1. Download or clone the **Flood Prediction** project from the GitHub repository.
2. Open the project folder in **Visual Studio Code** or any Python IDE.
3. Install the required dependencies using:

</> Bash

```
pip install -r requirements.txt
```

4. Start the Flask application by running:

</> Bash

```
python app.py
```

5. Open a web browser and visit:

```
http://127.0.0.1:5000
```

6. Enter the required weather parameters (such as annual rainfall, cloud visibility, and seasonal rain).
7. Click the **Predict** button to generate the flood prediction result.
8. The application will display the predicted flood risk based on the trained **XGBoost** model.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The system uses historical weather data and does not support real-time weather data.	Future versions will integrate real-time weather APIs for improved prediction accuracy.
2	The application is currently deployed on a local Flask server, limiting scalability.	Planned deployment on IBM Cloud to support more users and improve availability.
3	The system does not provide automatic SMS or email alerts after prediction.	Future enhancement includes notification services for early flood warnings.